PCE AutoBuy + Credit Scoring — Complete Integration Guide

> **What you're integrating:** Nightly credit scoring batch + real-time AutoBuy agent with LangGraph workflow.

Prerequisites

1. `migration_v2_to_v3_autobuy.sql` has been applied to your database
2. `AGENTS_ENABLED` flag is understood (AutoBuy is NOT an agent — it runs independently)
3. Payment gateway account (Razorpay recommended for India)

Step 1: Copy Files into Backend

```
cd ~/price_comparision_engine

# 1. Credit scoring tasks
cp /path/to/tasks_credit_scoring.py app/tasks_credit_scoring.py

# 2. AutoBuy agent
cp /path/to/tasks_autobuy.py app/tasks_autobuy.py

# 3. Update Celery app to include new modules
cp /path/to/celery_beat_complete_schedule.py app/celery_beat_schedule.py
```

Step 2: Update `app/celery_app.py`

2.1 Add imports

```
from app.config import settings  # Already present from feature flag
```

2.2 Update `include` list

```
include=[
    "app.tasks",
    "app.workers.scraper_worker",
    "app.workers.email_worker",
    "app.tasks_agents",
    "app.tasks_credit_scoring",   # NEW
    "app.tasks_autobuy",          # NEW
],
```

2.3 Update beat schedule (use the complete schedule from `celery_beat_complete_schedule.py`)

Key additions:

- `"credit-score-nightly-batch"` — runs `recalculate_all_credit_scores` daily

- `"autobuy-trigger-scan"` — runs `scan_auto_buy_triggers` every 5 minutes

Step 3: Update `app/config.py`

Add payment gateway settings:

```
# --- Payment Gateway (Razorpay) ---
RAZORPAY_KEY_ID: str = Field(default="")
RAZORPAY_KEY_SECRET: str = Field(default="")
RAZORPAY_WEBHOOK_SECRET: str = Field(default="")

# --- AutoBuy ---
AUTOBUY_ENABLED: bool = Field(default=False)  # Master switch
AUTOBUY_MAX_RETRIES: int = Field(default=3, ge=0, le=5)
AUTOBUY_PRICE_TOLERANCE_PCT: float = Field(default=5.0, ge=0, le=20)
```

Add to `.env`:

```
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxx
RAZORPAY_WEBHOOK_SECRET=whsec_xxxxxxxx
AUTOBUY_ENABLED=false
AUTOBUY_MAX_RETRIES=3
AUTOBUY_PRICE_TOLERANCE_PCT=5.0
```

> **Keep `AUTOBUY_ENABLED=false` until you're ready to launch.**

Step 4: Wire Payment Gateway (Razorpay)

4.1 Install SDK

```
pip install razorpay
```

4.2 Add to `requirements.txt`

```
razorpay==1.4.2
```

4.3 Replace mock functions in `tasks_autobuy.py`

Replace `_mock_payment_auth` and `_mock_vendor_order` with real implementations:

```
import razorpay

def _real_payment_auth(pm: Dict, amount: float) -> Dict:
    client = razorpay.Client(auth=(settings.RAZORPAY_KEY_ID, settings.RAZORPAY_KEY_SECRET))

    # Create payment using saved token
    resp = client.payment.create({
        "amount": int(amount * 100),  # paise
        "currency": "INR",
```

```
        "method": pm["method_type"],
        "token": pm["gateway_token"],
        "customer_id": pm["gateway_customer_id"],
        "capture": False,  # Authorize only
    })
    return resp

def _capture_payment(txn_id: str) -> None:
    client = razorpay.Client(auth=(settings.RAZORPAY_KEY_ID, settings.RAZORPAY_KEY_SECRET))
    client.payment.capture(txn_id, {"amount": None})  # Capture full amount

def _void_payment(txn_id: str) -> None:
    client = razorpay.Client(auth=(settings.RAZORPAY_KEY_ID, settings.RAZORPAY_KEY_SECRET))
    # Razorpay: refund the authorized amount without capture
    client.payment.refund(txn_id, {"amount": None, "speed": "normal"})
```

Step 5: Wire Vendor Order APIs

Each vendor requires a separate integration. Start with your highest-volume partners:

Vendor	API	Difficulty
Amazon India	Product Advertising API 5.0	Hard (requires approval)
Flipkart	Affiliate API	Medium
Direct merchants	Partner Feed API (your own)	Easy

For direct merchants using your Partner Feed:

```
def _place_partner_order(offer: Dict, addr: Dict, payment_token: str) -> Dict:
    # Call merchant's webhook_url from merchant_rules
    import requests
    resp = requests.post(
        offer["webhook_url"],
        json={
            "product_id": offer["vendor_product_id"],
            "quantity": 1,
            "shipping_address": addr,
            "payment_token": payment_token,
        },
        headers={"X-API-Key": offer["partner_api_key"]},
        timeout=30,
    )
    return resp.json()
```

Step 6: Add API Endpoints

Add to `app/api/`:

6.1 `app/api/autobuy.py` — User-facing endpoints

```python
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.core.auth import get_current_user

router = APIRouter(prefix="/api/v1/autobuy", tags=["AutoBuy"])

@router.post("/rules")
def create_auto_buy_rule(
    rule: AutoBuyRuleCreate,
    db: Session = Depends(get_db),
    user = Depends(get_current_user),
):
    """Create an AutoBuy rule for a product."""
    # Check credit eligibility
    credit = db.query(UserCreditProfile).filter_by(user_id=user.id).first()
    if not credit or not credit.auto_buy_eligible:
        raise HTTPException(403, "AutoBuy not enabled for your account")

    # Check consent
    consent = db.query(UserConsent).filter_by(
        user_id=user.id, consent_type="auto_buy"
    ).order_by(UserConsent.created_at.desc()).first()
    if not consent or not consent.consent_given:
        raise HTTPException(403, "AutoBuy consent required")

    # Create rule
    ...

@router.get("/rules")
def list_auto_buy_rules(user = Depends(get_current_user)):
    """List user's AutoBuy rules."""
    ...

@router.delete("/rules/{rule_id}")
def delete_auto_buy_rule(rule_id: str, user = Depends(get_current_user)):
    """Deactivate an AutoBuy rule."""
    ...

@router.get("/executions")
def list_auto_buy_executions(user = Depends(get_current_user)):
    """Show AutoBuy execution history."""
    ...
```

6.2 `app/api/payments.py` — Payment method management

```python
@router.post("/payment-methods")
def add_payment_method(
    method: PaymentMethodCreate,
    user = Depends(get_current_user),
):
    """Tokenize a payment method via Razorpay."""
    # Use Razorpay tokenization API
    # Store only gateway_token, never raw card data
    ...

@router.get("/payment-methods")
def list_payment_methods(user = Depends(get_current_user)):
    ...
```

Step 7: Flutter App Changes

7.1 AutoBuy Consent Screen

Add a consent screen before enabling AutoBuy:

```dart
// lib/screens/autobuy_consent_screen.dart
class AutoBuyConsentScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('AutoBuy Setup')),
      body: Padding(
        padding: EdgeInsets.all(20),
        child: Column(
          children: [
            Text('AutoBuy allows us to purchase products on your behalf when prices drop.'),
            SizedBox(height: 20),
            Text('Requirements:'),
            Text('• KYC verification completed'),
            Text('• Trust score >= 650'),
            Text('• Active payment method on file'),
            Text('• Shipping address saved'),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () => _grantConsent(context),
              child: Text('I Understand and Consent'),
            ),
          ],
        ),
      ),
    );
  }
}
```

7.2 AutoBuy Rule Creation

On product detail screen, add "Set AutoBuy" button:

```
// When user taps "Set AutoBuy"
showModalBottomSheet(
  context: context,
  builder: (_) => AutoBuyRuleSheet(
    product: product,
    onSave: (triggerPrice, maxValue) async {
      await api.createAutoBuyRule(
        productId: product.id,
        triggerPrice: triggerPrice,
        maxOrderValue: maxValue,
      );
    },
  ),
);
```

Step 8: Testing Checklist

8.1 Credit Scoring

```
# 1. Run batch manually
docker compose exec api celery -A app.celery_app call
↪  app.tasks_credit_scoring.recalculate_all_credit_scores

# 2. Check logs
docker compose logs -f celery_worker | grep CREDIT_BATCH

# 3. Verify scores
docker compose exec db psql -U price_engine -d price_comparison -c "
    SELECT user_id, trust_score, auto_buy_eligible FROM user_credit_profiles LIMIT 5;
"
```

8.2 AutoBuy (with mocks)

```
# 1. Create a test rule
curl -X POST http://localhost:8000/api/v1/autobuy/rules \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{
    "product_id": "...",
    "trigger_price": 5000,
    "max_order_value": 5000
  }'

# 2. Manually trigger scanner
docker compose exec api celery -A app.celery_app call app.tasks_autobuy.scan_auto_buy_triggers
```

```
# 3. Check execution logs
docker compose exec db psql -U price_engine -d price_comparison -c "
    SELECT rule_id, status, error_code, created_at
    FROM auto_buy_executions
    ORDER BY created_at DESC LIMIT 5;
"
```

8.3 End-to-End Flow

1. Register user → complete KYC → add payment method → add address
2. Wait for nightly batch (or manually trigger credit score update)
3. Verify `auto_buy_eligible = true` in `user_credit_profiles`
4. Grant AutoBuy consent
5. Create AutoBuy rule for a product
6. Manually lower product price in DB
7. Run `scan_auto_buy_triggers`
8. Verify `purchase_orders` table has new order
9. Verify `payment_transactions` has authorized payment
10. Verify `auto_buy_executions` shows `success` status

Step 9: Monitoring

9.1 Key Metrics

Metric	Query	Alert
Credit batch duration	`auto_buy_executions` grouped by hour	$>$ 30 min
AutoBuy success rate	`status = 'success' / total`	$<$ 85%
Credit gate rejections	`status = 'credit_check_failed'`	$>$ 20%
Payment failures	`status = 'payment_failed'`	$>$ 5%
Queue depth	Celery Flower dashboard	$>$ 100

9.2 Grafana Panels

Add to your Grafana dashboard:

- AutoBuy executions per hour (bar chart)
- Success rate % (gauge)
- Average trust score of AutoBuy users (stat)
- Payment failure reasons (pie chart)
- Top products by AutoBuy volume (table)

Step 10: Launch Checklist

- Database migration applied
- Razorpay account configured (test mode)
- `AUTOBUY_ENABLED=true` in `.env`
- Celery workers restarted
- Nightly batch tested
- AutoBuy trigger scanner tested
- Mock payment flow tested end-to-end
- Real payment flow tested with ₹1 transaction
- Flutter consent screen implemented
- Flutter AutoBuy rule creation tested
- Admin dashboard shows AutoBuy metrics
- Rollback plan documented

File Reference

File	Purpose	Where to Put
`tasks_credit_scoring.py`	Nightly batch + real-time score updates	`app/tasks_credit_scoring.py`
`tasks_autobuy.py`	AutoBuy agent with LangGraph workflow	`app/tasks_autobuy.py`
`celery_beat_complete_schedule.py`	Updated beat schedule	Merge into `app/celery_app.py`
`migration_v2_to_v3_autobuy.sql`	All new tables	Run once via `psql`
`AutoBuy_Agent_Architecture.md`	Full architecture doc	Reference only
`Credit_Rating_Algorithm.md`	Scoring algorithm deep-dive	Reference only